Shared Character State

Purpose

The Character State is the single source of truth describing the current observable condition of Shio.

Every Runtime contributes to the Character State.

Every Runtime observes the Character State.

Runtime systems never coordinate by directly controlling each other.

Instead, they coordinate through a shared understanding of the current state of the character.

This principle dramatically reduces coupling between Runtime systems while making the overall architecture easier to extend and maintain.

The Single Source of Truth

At any moment in time, there is only one current state of the character.

The Character State represents that state.

It does not belong to Speech Runtime.

It does not belong to Avatar Runtime.

It does not belong to the LLM.

It belongs to Shio.

Every Runtime may contribute information.

No Runtime owns the entire state.

Runtime Coordination

Runtime systems should not directly invoke each other's behavior whenever coordination can be expressed through the Character State.

For example:

```
Speech Runtime
        |
```

```
        ▼
Character State
        ▲
        │
Avatar Runtime
```

Instead of:

```
Speech Runtime
        │
        ▼
Avatar Runtime
```

This allows every interested subsystem to independently react to the same state transition.

A single state change may update:

- avatar animations;
- facial expressions;
- overlay indicators;
- debug panels;
- future integrations;
- logging;
- analytics.

without introducing new Runtime dependencies.

Observable Character Dimensions

The Character State is composed of several independent dimensions describing different aspects of the character.

Examples include:

- Runtime Mode
- Emotion
- Attention
- Speech
- Activity
- Avatar State
- Memory Context (future)
- Conversation Context (future)

Each dimension evolves independently while contributing to one coherent representation of the character.

Runtime Mode

Runtime Mode describes what Shio is currently doing.

Typical modes include:

- Idle
- Listening
- Thinking
- AwaitingSpeech
- Speaking
- Acting

Future versions of the Runtime may introduce additional modes without affecting existing Runtime boundaries.

Emotion

Emotion represents the currently active emotional state of the character.

Emotion is not owned by the LLM.

The LLM may suggest an emotion.

The Experience Layer may modify it.

System rules may override it.

Avatar Runtime renders it.

The Character State stores the final result.

Attention

Attention represents what currently has the character's focus.

Examples include:

- the current user;
- another participant;
- the live chat;
- an external event;
- no active target.

Attention influences future reactions but is not tied to any specific Runtime implementation.

Speech

Speech describes the communication state of the character.

Examples include:

- listening;
- waiting for recognition;
- generating a reply;
- speaking;
- silent.

Speech Runtime updates this information.

Avatar Runtime and Experience Layer consume it.

Activity

Activity represents the current high-level action performed by the character.

Examples include:

- answering;
- executing a tool;
- moderating chat;
- reacting to an event;
- waiting.

Activity is intentionally independent from visual animation.

One activity may be represented by multiple different animations.

Likewise, one animation may be reused by multiple activities.

Avatar State

Avatar State contains only the information required to render the current visual representation of the character.

Examples include:

- active body animation;
- current facial expression;
- active gesture;
- lipsync state;
- breathing state;
- idle controller state.

Avatar Runtime owns the visual implementation.

Character State owns the observable result.

Architectural Rule

Runtime systems never exchange implementation details.

Runtime systems exchange state.

Whenever possible, a Runtime should update the Character State rather than directly instruct another Runtime.

This principle keeps the architecture loosely coupled and allows new Runtime systems to be introduced without modifying existing ones.

Future Evolution

The Character State is expected to become the central coordination model of Shio Runtime.

Future Runtime systems such as:

- Memory Layer;
- Passive Companion Mode;
- External Service Adapters;
- Behavior Runtime;
- Scheduler Runtime;

should integrate through the Character State instead of introducing new direct Runtime dependencies.

As the project evolves, the Character State should remain the canonical representation of the digital character while individual Runtime implementations continue to evolve independently.